Guia prático: como integrar a API da OddsPapi no bot de apostas

Este documento foi feito para ajudar o Antigravity a colocar a OddsPapi para funcionar dentro de um bot de apostas, com foco em integração rápida, estrutura de código e prevenção de erros comuns.[1][1][2]

Visão geral

A OddsPapi oferece dados de odds esportivas em tempo real, incluindo odds pré-jogo, ao vivo, props, totais, spreads e histórico, com cobertura ampla de casas e esportes.[1] A documentação pública mostra que a API usa o host `https://api.oddspapi.io` e autentica as requisições por meio do parâmetro `apiKey` na query string.[1]

Na própria documentação pública, o fluxo inicial recomendado é: obter a chave da API, buscar torneios por esporte e depois consultar odds por torneios usando um bookmaker e uma lista de `tournamentIds`.[1] Já a documentação institucional também deixa claro que o serviço é B2B e pode exigir contato comercial para acesso completo, o que é importante considerar caso o bot ainda não tenha credenciais válidas ou acesso liberado.[2]

Fluxo mínimo de integração

A sequência mais simples para o bot funcionar é esta:[1]

1. Salvar a `apiKey` em variável de ambiente.
2. Consultar `/v4/tournaments?sportId=...&apiKey=...` para descobrir os torneios disponíveis de um esporte.[1]
3. Extrair os `tournamentId` desejados.[1]
4. Consultar `/v4/odds-by-tournaments?bookmaker=...&tournamentIds=...&apiKey=...` para receber

fixtures e mercados.[1]
5. Transformar a resposta em um formato interno do bot, por exemplo: evento, mercado, seleção, odd, horário e casa.
6. Aplicar a lógica do bot, como filtros de valor, alertas, arbitragem, monitoramento de mudança de linha ou montagem de entradas.

Endpoints mostrados na documentação

Endpoint	Uso	Observação
GET /v4/tournaments?sportId=10&apiKey=...	Lista torneios de um esporte	A documentação usa sportId=10 como exemplo.[1]
GET /v4/odds-by-tournaments?bookmaker=pinnacle&tournamentIds=17,8&apiKey=...	Retorna fixtures e odds por torneio	Exemplo oficial com bookmaker pinnacle.[1]

A documentação pública também mostra que o parâmetro oddsFormat pode ser usado para exibir odds em formato decimal ou americano.[1] Isso é útil porque bots de apostas normalmente funcionam melhor com uma padronização única, e o formato decimal costuma ser o mais simples para cálculo interno.[1]

Estrutura da resposta

No exemplo público de odds-by-tournaments, cada item retornado traz campos como fixtureId, sportId, tournamentId, startTime, updatedAt e um objeto bookmakerOdds com os mercados do bookmaker solicitado.[1] Dentro de bookmakerOdds, a documentação mostra uma estrutura aninhada com markets, depois outcomes, depois players, contendo campos como price, active, limit, changedAt e bookmakerOutcomeId.[1]

Na prática, isso significa que o bot não deve assumir uma resposta “achatada”; ele precisa percorrer objetos aninhados para encontrar cada odd utilizável.[1] Também vale tratar playerName como

opcional, porque o exemplo mostra casos em que esse campo vem null, especialmente em mercados simples como 1x2.[1]

Exemplo em Node.js

Abaixo está um exemplo simples de integração em Node.js usando fetch, ideal para um bot em JavaScript ou TypeScript:

```
const API_KEY = process.env.ODDSPAPI_KEY;
const BASE_URL = 'https://api.oddspapi.io';

async function getTournaments(sportId) {
  const url = `${BASE_URL}/v4/tournaments?sportId=${sportId}&apiKey=${AP
  const res = await fetch(url);

  if (!res.ok) {
    throw new Error(`Erro ao buscar torneios: ${res.status} ${res.statusText}`
  }

  return res.json();
}

async function getOddsByTournaments({ bookmaker, tournamentIds, odd
  const ids = tournamentIds.join(',');
  const url = `${BASE_URL}/v4/odds-by-tournaments?bookmaker=${encodeUR
  const res = await fetch(url);

  if (!res.ok) {
    throw new Error(`Erro ao buscar odds: ${res.status} ${res.statusText}`);
  }

  return res.json();
}

function normalizeOdds(fixtures) {
  const rows = [];

  for (const fixture of fixtures) {
```

```
    const bookmakerEntries = Object.entries(fixture.bookmakerOdds || {});

    for (const [bookmaker, bookmakerData] of bookmakerEntries) {
      const markets = bookmakerData?.markets || {};

      for (const [marketId, marketData] of Object.entries(markets)) {
        const outcomes = marketData?.outcomes || {};

        for (const [outcomeId, outcomeData] of Object.entries(outcomes)) {
          const players = outcomeData?.players || {};

          for (const [playerKey, playerData] of Object.entries(players)) {
            rows.push({
              fixtureId: fixture.fixtureId,
              sportId: fixture.sportId,
              tournamentId: fixture.tournamentId,
              startTime: fixture.startTime,
              updatedAt: fixture.updatedAt,
              bookmaker,
              marketId,
              outcomeId,
              playerKey,
              bookmakerOutcomeId: playerData.bookmakerOutcomeId,
              playerName: playerData.playerName,
              price: playerData.price,
              active: playerData.active,
              limit: playerData.limit,
              changedAt: playerData.changedAt,
            });
          }
        }
      }
    }
  }

  return rows;
}
```

```
async function main() {
  const tournaments = await getTournaments(10);
  const selectedTournamentIds = tournaments.slice(0, 3).map(t => t.tournam

  const fixtures = await getOddsByTournaments({
    bookmaker: 'pinnacle',
    tournamentIds: selectedTournamentIds,
    oddsFormat: 'decimal',
  });

  const normalized = normalizeOdds(fixtures);
  console.log(normalized.slice(0, 10));
}

main().catch(console.error);
```

Esse exemplo segue exatamente o padrão exposto na documentação pública: primeiro busca torneios por `sportId`, depois consulta odds por `tournamentIds` e bookmaker.[1] Ele também resolve o principal problema de integração: transformar a estrutura profundamente aninhada em linhas simples que o bot consegue filtrar e comparar.[1]

Exemplo em Python

Se o bot do Antigravity estiver em Python, este esqueleto resolve o básico:

```
import os
import requests

API_KEY = os.getenv("ODDSPAPI_KEY")
BASE_URL = "https://api.oddspapi.io"


def get_tournaments(sport_id: int):
    url = f"{BASE_URL}/v4/tournaments"
    params = {
        "sportId": sport_id,
```

```
        "apiKey": API_KEY,
    }
    r = requests.get(url, params=params, timeout=20)
    r.raise_for_status()
    return r.json()


def get_odds_by_tournaments(bookmaker: str, tournament_ids: list[int], od
    url = f"{BASE_URL}/v4/odds-by-tournaments"
    params = {
        "bookmaker": bookmaker,
        "tournamentIds": ",".join(map(str, tournament_ids)),
        "oddsFormat": odds_format,
        "apiKey": API_KEY,
    }
    r = requests.get(url, params=params, timeout=20)
    r.raise_for_status()
    return r.json()


def normalize_odds(fixtures):
    rows = []

    for fixture in fixtures:
        bookmaker_odds = fixture.get("bookmakerOdds", {})

        for bookmaker, bookmaker_data in bookmaker_odds.items():
            markets = bookmaker_data.get("markets", {})

            for market_id, market_data in markets.items():
                outcomes = market_data.get("outcomes", {})

                for outcome_id, outcome_data in outcomes.items():
                    players = outcome_data.get("players", {})

                    for player_key, player_data in players.items():
                        rows.append({
                            "fixtureId": fixture.get("fixtureId"),
```

```
                    "sportId": fixture.get("sportId"),
                    "tournamentId": fixture.get("tournamentId"),
                    "startTime": fixture.get("startTime"),
                    "updatedAt": fixture.get("updatedAt"),
                    "bookmaker": bookmaker,
                    "marketId": market_id,
                    "outcomeId": outcome_id,
                    "playerKey": player_key,
                    "bookmakerOutcomeId": player_data.get("bookmakerOutcom
                    "playerName": player_data.get("playerName"),
                    "price": player_data.get("price"),
                    "active": player_data.get("active"),
                    "limit": player_data.get("limit"),
                    "changedAt": player_data.get("changedAt"),
                })

    return rows


def main():
    tournaments = get_tournaments(10)
    selected_ids = [t["tournamentId"] for t in tournaments[:3]]
    fixtures = get_odds_by_tournaments("pinnacle", selected_ids)
    normalized = normalize_odds(fixtures)
    print(normalized[:10])


if __name__ == "__main__":
    main()
```

A lógica é a mesma do exemplo em JavaScript porque a estrutura da API não muda: autenticação por `apiKey`, host `api.oddspapi.io`, torneios por esporte e odds por torneios.[1] O ponto central não é a linguagem, mas a normalização dos mercados para um modelo que o bot entenda bem.[1]

Como encaixar isso no bot

A melhor forma de plugar a OddsPapi em um bot é separar a integração em quatro camadas:

- **Camada de configuração**: guarda `ODDSPAPI_KEY`, bookmaker padrão, esportes monitorados e intervalo de atualização.
- **Camada cliente**: concentra chamadas HTTP para a OddsPapi, sempre usando `https://api.oddspapi.io` e `apiKey`.[1]
- **Camada de normalização**: transforma `bookmakerOdds -> markets -> outcomes -> players` em registros simples.[1]
- **Camada de estratégia**: decide quando alertar, apostar, comparar linhas ou identificar oportunidade.

Um modelo interno útil seria este:

```
interface OddRow {
  fixtureId: string;
  sportId: number;
  tournamentId: number;
  bookmaker: string;
  marketId: string;
  outcomeId: string;
  bookmakerOutcomeId: string | null;
  playerName: string | null;
  price: number | null;
  active: boolean;
  limit: number | null;
  startTime: string | null;
  changedAt: string | null;
  updatedAt: string | null;
}
```

Esse formato já conversa melhor com filtros, logs, banco de dados e comparações históricas do que a resposta original da API.[1] Ele também facilita criar regras como “avisar quando a odd subir acima de 2.10” ou “ignorar mercados inativos”.[1]

Erros comuns

Os problemas mais prováveis nessa integração são estes:

- Colocar a chave em header em vez de mandar como parâmetro `apiKey`, embora a documentação pública mostre autenticação por query string.[1]
- Assumir que a resposta vem pronta em lista de odds simples, quando na prática ela vem aninhada em bookmaker, mercado, outcome e player.[1]
- Não tratar campos opcionais como `playerName` ou estruturas que podem variar entre mercados.[1]
- Misturar formatos de odds sem padronização, mesmo a documentação citando o uso de `oddsFormat` para decimal ou americano.[1]
- Esquecer que o acesso pode ser comercial/B2B e, por isso, testar com credenciais incompletas ou sem permissão suficiente.[2]

Checklist para o Antigravity

Antes de dizer que “a API não funciona”, vale conferir esta lista:

- A chave `ODDSPAPI_KEY` está carregada no ambiente.
- A URL base usada é `https://api.oddspapi.io`.[1]
- As chamadas incluem `apiKey` nos parâmetros.[1]
- O `sportId` usado realmente retorna torneios.[1]
- Os `tournamentIds` foram montados como string separada por vírgulas.[1]
- O `bookmaker` usado é aceito pela conta e pelo endpoint.[1]
- O bot consegue percorrer `bookmakerOdds.markets.outcomes.players` sem quebrar.[1]
- O parser ignora odds inativas e aceita campos nulos.[1]

Exemplo de estratégia simples

Um primeiro caso de uso funcional é monitorar partidas futuras de um esporte específico, pedir odds de um bookmaker e disparar alerta quando uma odd ativa ultrapassar um valor mínimo definido pelo usuário.[1] Como o retorno inclui `changedAt`, `updatedAt`, `limit` e `price`,

o bot pode registrar mudança de linha, filtrar liquidez mínima e evitar alertas repetidos em odds antigas.[1]

Recomendação prática

Para fazer a integração funcionar rápido, o caminho mais seguro é começar só com um esporte, um bookmaker e um único endpoint de odds por torneio, deixando WebSocket, múltiplas casas e mercados avançados para a segunda etapa.[1][2] A documentação pública já entrega o suficiente para montar esse MVP: autenticação por apiKey, descoberta de torneios e consulta de odds por tournamentIds.[1]

Próximo passo sugerido

O próximo passo técnico ideal é pedir ao Antigravity para substituir o parser atual do bot por uma função de normalização como a dos exemplos acima e testar uma chamada real com logs completos de URL, status HTTP e primeiros itens do JSON.[1] Isso normalmente mostra em poucos minutos se o problema está na credencial, na montagem da URL ou na leitura incorreta da estrutura retornada.[1]